

Deep Learning – Advanced Study Assignment

Why Does the Direction of KL Divergence Lead to Different Optimization Behaviors and Modeling Outcomes?

A_02 Report · April 2026

Research Question	Why does the direction of KL divergence (forward vs. reverse) lead to different optimization behaviors and modeling outcomes?
Keywords	KL divergence, entropy, cross-entropy, mean-seeking, mode-seeking, bimodal distribution, mean, variance
Tools	PyTorch, NumPy, Matplotlib, AI tools (GPTs), Wikipedia, Articles
Dataset	Synthetic bimodal Gaussian mixture (toy experiment)

Abstract

The Kullback–Leibler divergence (KL divergence) is a fundamental quantity in machine learning, deep learning, and information theory, widely used to measure the difference between probability distributions. Despite its central role in probabilistic modelling, its inherent asymmetry is often overlooked in practical applications. This asymmetry arises from the direction of evaluation, specifically from which distribution defines the expectation: forward KL divergence $D_{KL}(P||Q)$ evaluates discrepancies under the target distribution, while reverse KL divergence $D_{KL}(Q||P)$ evaluates them under the model distribution, leading to fundamentally different optimization behaviours.

This study investigates how the direction of KL divergence affects optimization dynamics and modeling outcomes. A theoretical analysis based on gradient behavior is combined with controlled experiments in which a unimodal Gaussian model is trained to approximate a bimodal target distribution. The experiments systematically examine the effects of initialization, learning rate, and mode separation.

The results demonstrate that forward KL divergence consistently produces mode-covering behaviour, characterized by higher entropy, greater stability, and robustness to initialization. In contrast, reverse KL divergence exhibits mode-seeking behavior, leading to lower entropy, sensitivity to initialization, and convergence toward local modes. These differences become more pronounced as the complexity and separation of the target distribution increase.

The findings highlight that the direction of KL divergence is not a minor implementation detail but a critical design choice that governs whether a model prioritizes distributional coverage or concentrated approximation.

1. Introduction and Motivation

The comparison of probability distributions is a central problem in machine learning, deep learning, statistics, and information theory. Among the most widely used measures for quantifying divergence between distributions is the Kullback–Leibler divergence (KL divergence), which plays a fundamental role in learning algorithms ranging from maximum likelihood estimation to variational inference and deep generative modelling.

Despite its broad applicability, KL divergence possesses an important but often underemphasized property: **asymmetry**. Specifically, the divergence between two probability distributions $P(x)$ and $Q(x)$ depends on the direction of evaluation, such that:

$$D_{KL}(P || Q) \neq D_{KL}(Q || P)$$

This asymmetry implies that the choice of direction is not merely a mathematical formality, but a decision that can significantly influence learning behaviour, optimization dynamics, and model bias¹.

In modern deep learning systems, KL divergence appears in multiple contexts, including variational autoencoders, reinforcement learning policy updates, and probabilistic inference frameworks such as Variational inference. However, the practical consequences of selecting either forward or reverse KL divergence are often treated implicitly rather than explicitly analyzed.

In this study I would like to systematically investigate the influence of KL divergence direction on optimization outcomes and learned representations. Using controlled synthetic experiments implemented in **PyTorch**, then I will analyze how forward and reverse KL divergences behave under varying initialization conditions, learning rates, and target distribution complexities.

The primary objective of this work is to answer the following question:

“How does the direction of KL divergence influence the optimization behavior, and Modeling Outcomes?”

As it is hypothesized that forward KL divergence encourages mode-covering behavior and higher uncertainty, whereas reverse KL divergence promotes mode-seeking behavior and sharper, more concentrated distributions.

Why the question is interesting?

In my first report, I found entropy and cross-entropy very closely linked to the KL divergence. But to fully develop my intuition, I would like to cover all other aspects of KL divergence, why it is related to entropy, cross-entropy and why it has two different directions.

How it is connected to the lecture material?

In our lecture material, many aspects are already covered, and I could follow the direction of the materials covered. However, this assignment is purely given to us to develop the critical thinking and the flow of Deep Learning, where we should understand it is not about listening to the lecture but taking time for deep research and making more self-disciplined study.

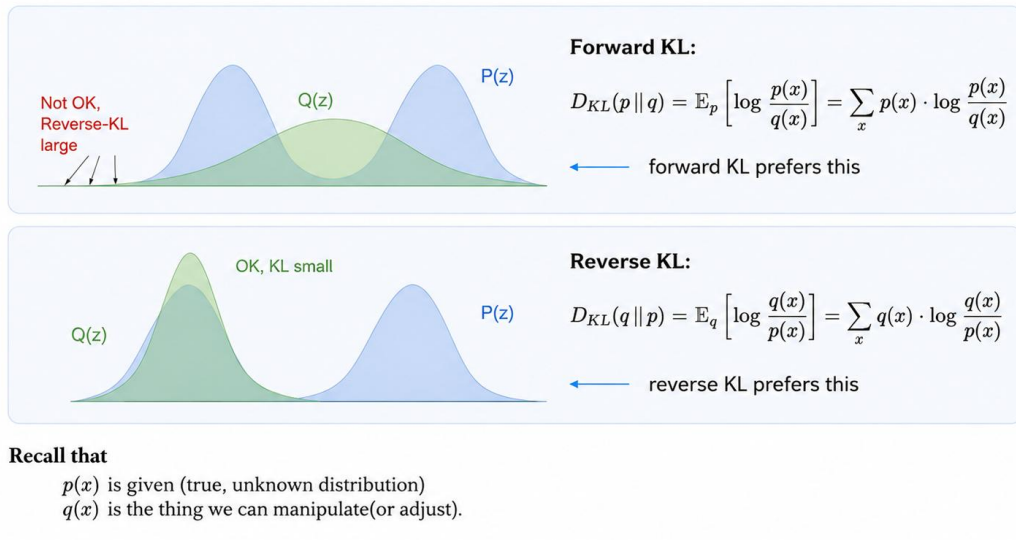
And Why you chose it?

As I already stated that the investigation of this topic is more about developing my intuition and clean understanding of Deep Learning, where I can connect and clearly understand the mechanism of KL divergence, while it gives me time to read and develop the theory behind, get well use of available resources.

One example where in this slide, I would like to be quietened myself how and why we need such an approach, and where such intuition should be revoked, or how we know when to use forward or reverse, and more other questions lead me to make this report.

Recall: Forward KL vs. Reverse KL

- Forward-KL: Q in order to **cover every modes** of P
- Reverse KL: Q covers only **one mode** of P



Now let's start to deep dive into the theoretical part and then to experimental.

2. Background and Theoretical Framework

2.1 Information-Theoretic Foundations²

KL divergence arises naturally from information theory. The entropy of a distribution $P(x)$ measures the optimal coding length:

$$H(P) = -\mathbb{E}_P[\log P(x)]$$

When we approximate $P(x)$ using another distribution $Q(x)$, the expected coding cost becomes the cross-entropy:

$$H(P, Q) = -\mathbb{E}_P[\log Q(x)]$$

The KL divergence is defined as the difference between these two quantities:

$$D_{KL}(P \parallel Q) = H(P, Q) - H(P)$$

Thus, KL divergence quantifies the additional cost (extra coding length) added when using Q to approximate P .

2.2 KL Divergence Definition²

The **Kullback–Leibler (KL) divergence** is a measure of distance between two probability distributions $P(x)$ and $Q(x)$, defined as:

$$D_{KL}(P||Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right]$$

Where:

- $P(x)$: true data distribution
- $Q(x)$: model distribution

It is important to note that KL divergence is not a true distance metric, as it is not symmetric and does not satisfy the triangle inequality. Instead, it should be interpreted as a measure of discrepancy between two distributions.

2.3 Forward vs Reverse KL

2.3.1 Forward KL Formulation ($P \parallel Q$):

Forward KL computes the expectation over the true data distribution P .

The formula is mathematically represented as:

$$D_{KL}(P||Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right] = \int P(x) \log \frac{P(x)}{Q(x)} dx$$

- The integral form applies to **continuous distributions**; for discrete cases, you'd use a sum:

$$\sum_x P(x) \log \frac{P(x)}{Q(x)}$$

- It assumes **support matching**, meaning: if $P(x) > 0$, then $Q(x) > 0$, otherwise the KL divergence becomes infinite.

2.3.2 Reverse KL Formulation ($Q \parallel P$):

Reverse KL computes the expectation over the model's predicted distribution Q .

The formula is:

$$D_{KL}(Q||P) = \mathbb{E}_{x \sim Q} \left[\log \frac{Q(x)}{P(x)} \right] = \int Q(x) \log \frac{Q(x)}{P(x)} dx$$

Forward KL vs. Reverse KL

- $KL(p,q) \neq KL(q,p) \rightarrow$ then how they are different?

P : true Q : guess

Forward KL : $KL(p||q) = \mathbb{E}_p \left[\log \frac{p}{q} \right] = \sum_x p(x) \cdot \log \frac{p(x)}{q(x)}$

Reverse KL : $KL(q||p) = \mathbb{E}_q \left[\log \frac{q}{p} \right] = \sum_x q(x) \cdot \log \frac{q(x)}{p(x)}$

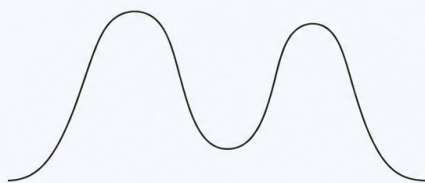
Recall that

$p(x)$ is given (true, unknown distribution)

$q(x)$ is the thing we can manipulate(or adjust).

Forward KL vs. Reverse KL

- $KL(p,q) \neq KL(q,p) \rightarrow$ then how they are different?



$p(x)$: true distribution



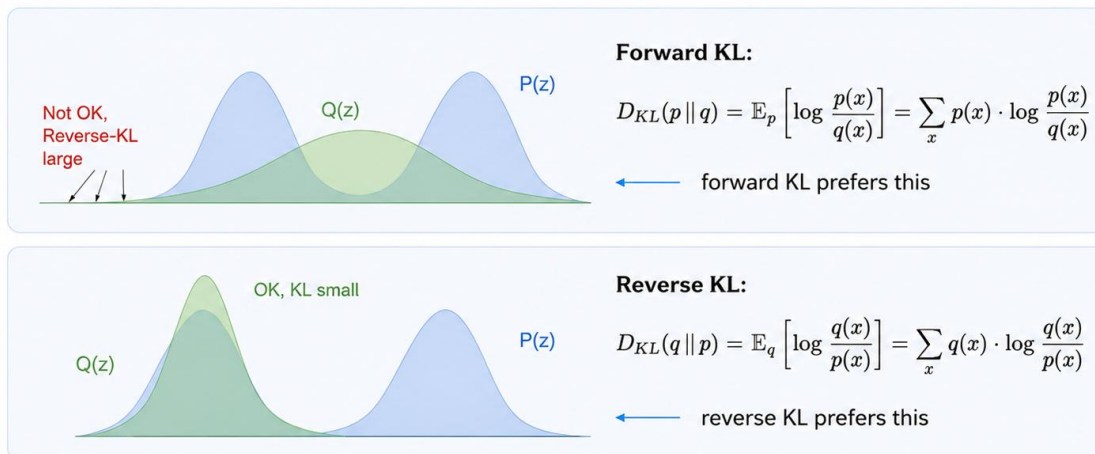
$q(x)$: guess (Gaussian)

We can adjust this by
changing mean & variance)

By minimizing $KL(p||q)$ or $KL(q||p)$,
what will be results like?

Recall: Forward KL vs. Reverse KL

- Forward-KL: Q in order to **cover every modes** of P
- Reverse KL: Q covers only **one more** of P



Recall that

$p(x)$ is given (true, unknown distribution)
 $q(x)$ is the thing we can manipulate (or adjust).

2.4 KL divergence is not symmetric ²⁻⁶

A key property of the Kullback–Leibler (KL) divergence is its asymmetry:

$$D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$$

As we know already KL divergence formulation lets go step by step to the explanation of asymmetry:

2.4.1 Formal Explanation of the Source of Asymmetry

At first glance, the two divergences

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}, \quad D_{KL}(Q \parallel P) = \sum_i Q(i) \log \frac{Q(i)}{P(i)}$$

appear structurally similar. However, exchanging P and Q induces **two simultaneous and non-equivalent changes**:

1. Change in the Weighting Measure

The KL divergence is an **expectation with respect to a specific distribution**:

$$D_{KL}(P\|Q) = \mathbb{E}_{i \sim P} \left[\log \frac{P(i)}{Q(i)} \right]$$

$$D_{KL}(Q\|P) = \mathbb{E}_{i \sim Q} \left[\log \frac{Q(i)}{P(i)} \right]$$

Thus, exchanging P and Q changes the **underlying measure of integration (or summation)**:

- In $D_{KL}(P\|Q)$, outcomes are weighted according to their probability under P
- In $D_{KL}(Q\|P)$, outcomes are weighted according to their probability under Q

Since $P \neq Q$ in general, these expectations are taken over **different probability spaces**, which already prevents symmetry.

2. Change in the Logarithmic Term

Simultaneously, the argument of the logarithm is inverted:

$$\log \frac{P(i)}{Q(i)} \longrightarrow \log \frac{Q(i)}{P(i)} = -\log \frac{P(i)}{Q(i)}$$

While this introduces a sign change at the level of the integrand, it does **not** produce a simple negation of the divergence because the weighting distribution also changes.

$$D_{KL}(P\|Q) - D_{KL}(Q\|P)$$

Substitute:

$$= \sum_i P(i) \log \frac{P(i)}{Q(i)} - \sum_i Q(i) \log \frac{Q(i)}{P(i)}$$

Use the log identity again:

$$\begin{aligned} &= \sum_i P(i) \log \frac{P(i)}{Q(i)} + \sum_i Q(i) \log \frac{P(i)}{Q(i)} \\ &= \sum_i (P(i) - Q(i)) \log \frac{P(i)}{Q(i)} \end{aligned}$$

This asymmetry can be understood intuitively by interpreting KL divergence as a measure of “**surprise**”. Specifically, it quantifies how surprising it is to observe samples from one distribution when they are evaluated under another distribution.

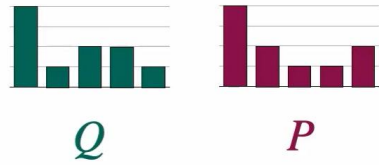
More formally, $D_{KL}(P\|Q)$ measures how well the distribution Q explains samples drawn from P . Reversing the arguments changes the sampling process, and therefore changes the notion of “surprise.” As a result, the two quantities are fundamentally different.

2.3.1 Conceptual intuition.

KL divergence measures "surprise." Specifically, it asks: if our sample points from one distribution (P), how surprised would we be if those points actually modelled a different distribution (Q)? Because sampling from P to get Q is a different process than sampling from Q to get P , the resulting "surprise" values are different.

I would like to give some illustration examples bellow.

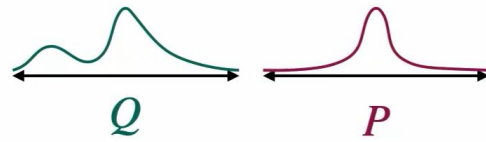
KL Divergence



$$\mathbb{D}_{KL}(P||Q) = \sum_i p_i \log \left(\frac{p_i}{q_i} \right)$$

$$\mathbb{D}_{KL}(Q||P) = \sum_i q_i \log \left(\frac{q_i}{p_i} \right)$$

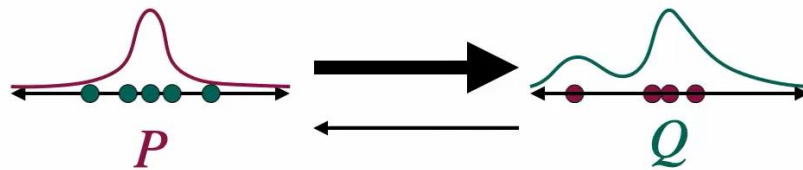
Not the same!



$$\mathbb{D}_{KL}(P||Q) = \int_{\mathbb{R}} p_i \log \left(\frac{p_i}{q_i} \right)$$

$$\mathbb{D}_{KL}(Q||P) = \int_{\mathbb{R}} q_i \log \left(\frac{q_i}{p_i} \right)$$

Not the same!



$\mathbb{D}_{KL}(P||Q)$: If I sample from P , how surprised am I if I get Q ?

$\mathbb{D}_{KL}(Q||P)$: If I sample from Q , how surprised am I if I get P ?

Simple Illustration:



small $\mathbb{D}_{KL}(P||Q)$: If I sample from P , how surprised am I if I get Q ?

Large

$\mathbb{D}_{KL}(Q||P)$: If I sample from Q , how surprised am I if I get P ?

The Salad vs. Tomato Analogy

P is a salad and Q is a bag of tomatoes.

- **KL (P, Q):** If our sample from a salad, we aren't very surprised to pull out a tomato. This results in a small KL divergence.
- **KL (Q, P):** If our sample from a bag of tomatoes, we would be extremely surprised to pull out a full salad (lettuce, onions, dressing). This results in a large KL divergence. KL Divergence is not a geometric distance; it is a measure of how easily one distribution can represent or generate another. Because the direction of this transformation matters, the symmetry does not hold.

2.5. Strengths and Limitations

The choice between forward and reverse KL divergence is rarely a matter of one being "better" than the other; rather, it is a choice of which type of error the practitioner is willing to tolerate.

2.5.1. Forward KL ($D_{KL}(P \parallel Q)$)

Strengths	Limitations
Global Coverage: Guarantees that every mode of the target distribution is represented. It is "all-encompassing."	Over-generalization: Often places high probability mass in "empty" regions (valleys) where the true probability is zero.
Initialization Robustness: The optimization landscape is generally smoother, making it less likely to get stuck in poor local minima.	Blurry Approximations: In high-dimensional spaces (like images), this leads to "blurry" results because the model tries to average all possible outcome
Entropy Maximization: Naturally captures the full variance and uncertainty of the data.	Computationally Expensive: Requires sampling from the true distribution P, which is often unknown or represented only by a finite dataset.

2.5.2. Reverse KL ($D_{KL}(Q \parallel P)$)

Strengths	Limitations
High Precision: The model produces very "realistic" samples that are guaranteed to stay within the high-probability regions of the target.	Mode Collapse: Completely ignores parts of the data. If the target has 10 modes, the model might only learn one and "forget" the other nine.
Computational Efficiency: Does not require samples from the true P ; it only needs to be able to evaluate the density $P(x)$, making it ideal for Variational Inference.	Initialization Sensitivity: Highly dependent on where the model starts. A poor initialization can lead to a "dead" model that never finds the target mass.
Sharp Representations: Produces low-entropy, concentrated distributions that are useful for decision-making and optimization.	Underestimates Uncertainty: Often leads to "overconfidence," where the model is very certain about a narrow range of values while ignoring others.

We can also frame this using the "**Zero-Avoiding**" vs. "**Zero-Forcing**" terminology:

1. **Forward KL is "Zero-Avoiding"**: It avoids assigning a probability of zero to any region where the target is non-zero ($Q(x) \neq 0$ if $P(x) > 0$).

Strength: We don't miss anything.

Limitation: We get a lot of "noise" or "blur."

2. **Reverse KL is "Zero-Forcing"**: It forces the model to be zero wherever the target is zero ($Q(x) = 0$ if $P(x) = 0$).

Strength: We get very "clean" and "accurate" samples.

Limitation: We lose the "big picture" of the data.

2.6 Gradient Behavior Comparison and Training Signal:

The fundamental difference between the two loss functions lies in how they penalize errors during optimization.

- **Forward KL is "Zero-Avoiding" (Mean-Seeking):**

Looking at the ratio $\frac{P(x)}{Q(x)}$. If the true data $P(x) > 0$ but the model predicts $Q(x) \rightarrow 0$, the ratio approaches infinity, resulting in a massive penalty. Therefore, the model Q is forced to stretch out and cover *every* region where true data exists. If the true data has two distinct peaks (bimodal) and our model can only create one peak (unimodal), the model will place its broad peak right in the middle to cover both. This leads to safe, but "blurry" or averaged predictions.

- **Reverse KL is "Zero-Forcing" (Mode-Seeking):**

Looking at the ratio $\frac{Q(x)}{P(x)}$ in the Reverse KL formula. If the model predicts $Q(x) > 0$ in a region where the true data $P(x) \rightarrow 0$, the penalty explodes. To avoid this, the model Q must be exactly zero wherever P is zero. Faced with bimodal true data, a unimodal model cannot stretch across the empty space between the peaks. Instead, it will collapse and tightly fit onto just *one* of the peaks, completely ignoring the other. This causes "mode collapse."

2.7 Practical Implications

The theoretical "mode-covering" vs. "mode-seeking" behavior translates into specific design outcomes in modern AI systems.

2.7.1 Generative Modeling: Diversity vs. Fidelity

In generative AI, there is a constant tension between **diversity** (showing all possible variations) and **fidelity** (how realistic each sample looks).

Forward KL (Likelihood-based models): Models like Autoregressive models or Variational Autoencoders (VAEs) that prioritize likelihood tend to be "diverse." They will try to cover every image in the training set. However, because they are "mode-covering," they often produce blurry results where the model averages out different possibilities.

Reverse KL (Adversarial/Variational Inference): Many GAN-based approaches or specific VI frameworks behave like reverse KL. They produce incredibly sharp, high-fidelity images. The trade-off is "**Mode Collapse**"—the model might only learn to draw three types of perfect faces and completely "forget" that other ethnicities or lighting conditions exist.

2.7.2 Variational Inference (VI): The "Overconfidence" Problem

In Bayesian deep learning, we use Reverse KL because we usually can't sample from the true posterior P .

The Implication: Because Reverse KL is "zero-forcing," the resulting model is almost always **under-dispersed**.

The Risk: The model will be "overconfident." It might give a prediction with 99% certainty because it has collapsed onto one local mode of the truth, completely ignoring other valid interpretations of the data. This is dangerous in high-stakes fields like **autonomous driving** or **medical prognosis**, where knowing what you *don't* know is critical.

2.7.4 Reinforcement Learning (RL): Exploration vs. Exploitation

In RL algorithms like **Proximal Policy Optimization (PPO)** or **Soft Actor-Critic (SAC)**, KL divergence is used to ensure the "new" policy doesn't stray too far from the "old" policy.

Forward KL Implication: Encourages the agent to keep its options open. It prevents the policy from prematurely committing to one path, maintaining better **exploration**.

Reverse KL Implication: Encourages the agent to refine a specific successful strategy. This leads to faster **exploitation** and convergence but risks the agent getting stuck in a sub-optimal "local" strategy (a local mode) and never finding the global best.

3. Experimental Verification

3.1 Experimental Setup

Classic experiment comparing Forward KL (Mean-Seeking) and Reverse KL (Mode-Seeking) by fitting a single Gaussian $Q(x)$ to a bimodal Gaussian mixture $P(x)$.

To investigate why the direction of Kullback–Leibler (KL) divergence leads to different optimization behaviors, we construct a controlled synthetic experiment. The key idea is to intentionally create a **model mismatch scenario**, where the model is not expressive enough to perfectly represent the true data distribution. This forces the optimization process to reveal its inherent bias.

Specifically, we approximate a **bimodal target distribution** using a **unimodal model**. Because the model cannot capture both modes simultaneously, the optimization objective must decide how to allocate probability mass. This provides a clear setting to observe how forward and reverse KL divergence behave differently.

Conceptual Clarification of the Problem

To avoid ambiguity, we explicitly define the key components of the problem:

3.1.1 Target Distribution $P(x)$

The true data-generating distribution $P(x)P(x)P(x)$ is defined as a symmetric mixture of two Gaussian components:

$$P(x) = 0.5 \cdot \mathcal{N}(x \mid -\mu, 1) + 0.5 \cdot \mathcal{N}(x \mid \mu, 1)$$

- $\mathcal{N}(x|\mu, \sigma^2)$ denotes a Gaussian density
- the distribution has **two symmetric modes** at $x=-\mu$ and $x=\mu$
- Variance of each component is fixed to 1
- the parameter $\mu>0$ controls **mode separation**

The equal weighting (0.5, 0.5) ensures perfect symmetry, meaning neither mode is preferred. Fixing the variance to 1 isolates μ as the only factor controlling task difficulty

Interpretation:

- Small μ : strong overlap $\rightarrow$ easier approximation
- Large μ : well-separated modes $\rightarrow$ harder approximation

Important

1. Why the 0.5 (The Mixing Weights)

The 0.5 values represent the **weight** or **probability** of each of the two Gaussian distributions in the mixture.

Symmetry: By setting both weights to 0.5, it ensures the two "peaks" (modes) of the distribution are exactly the same height. Neither side is heavier or more likely than the other.

Law of Probability: In a valid probability distribution, the total probability must sum to 1. Since there are two distributions and we want them to be equal, we divide that probability in

half ($0.5 + 0.5 = 1$). This means a data point has a 50% chance of coming from the left distribution and a 50% chance of coming from the right one.

2. Why the 1 (The Variance)

The 1 inside the normal distribution notation $\mathcal{N}(\mu, \sigma^2)$ represents the **variance** (σ^2).

Constant Width: Setting the variance to 1 fixes the "width" or "fatness" of both bell curves to a standard size.

Isolating the Difficulty: Because the widths are locked at a constant 1, the *only* thing that determines how much the two distributions overlap is the distance between them (controlled by μ). If the variance were allowed to change, the overlap would depend on a messy combination of both distance (μ) and width (σ^2). Keeping the variance at 1 isolates μ as the single "dial" you can turn to make the approximation harder or easier.

3.1.2 Model Distribution Q(x)

The approximating model $Q(x)$ is defined as:

$$Q(x) = \mathcal{N}(x \mid \mu_q, \sigma_q^2)$$

where both mean μ_q and variance σ_q^2 are learnable parameters

Because $Q(x)$ is unimodal, it cannot represent both modes of $P(x)$, forcing the optimization process to choose how to approximate the target distribution.

3.1.3 KL Divergence Objectives

We compare two optimization objectives:

Forward KL

$$D_{KL}(P \parallel Q) = \mathbb{E}_{x \sim P}[\log P(x) - \log Q(x)]$$

Since $\log P(x)$ does not depend on the model parameters, minimizing forward KL reduces to:

$$\min -\mathbb{E}_{x \sim P}[\log Q(x)]$$

This corresponds to maximizing likelihood under samples drawn from $P(x)$.

Reverse KL

$$D_{KL}(Q \parallel P) = \mathbb{E}_{x \sim Q}[\log Q(x) - \log P(x)]$$

In this case, the expectation is taken with respect to the model distribution $Q(x)$, making the optimization dependent on where the model currently assigns probability mass.

3.1.4 Implementation

The experiment is implemented in PyTorch. Sampling from the target distribution is performed explicitly by first selecting one of the two Gaussian components with equal probability and then drawing from that component.

Implementation of $P(x)$

```
# Sample from bimodal P(x)
def sample_from_p(mu, n=1000):
    mix = torch.bernoulli(0.5 * torch.ones(n))
    means = mu * (2 * mix - 1)
    return torch.randn(n) + means
```

Explanation of the Code

- A Bernoulli random variable selects between the two Gaussian components.
- If the sampled value is 0 $\rightarrow$ mean = $-\mu$
- If the sampled value is 1 $\rightarrow$ mean = $+\mu$
- A standard normal sample is then shifted by the selected mean.

This process exactly matches the mathematical definition of the mixture distribution.

Log-Density Implementation

```
# Log P(x)
def log_p(x, mu):
    comp1 = torch.exp(-0.5 * (x + mu)**2) / np.sqrt(2*np.pi)
    comp2 = torch.exp(-0.5 * (x - mu)**2) / np.sqrt(2*np.pi)
    return torch.log(0.5 * (comp1 + comp2) + 1e-10)

# Log Q(x)
def log_q(x, mu_q, var_q):
    return -0.5 * torch.log(2*np.pi*var_q) - (x - mu_q)**2 / (2*var_q)
```

These functions compute the log-probability under the model and the target distribution.

Optimization Procedure

The model parameters μ and σ_q are optimized using gradient descent.

```
def run_once(kl_type='forward', mu_target=2.0, init_mu_q=0.0, lr=0.05, steps=1000):
    mu_q = torch.tensor([init_mu_q], requires_grad=True)
    log_sigma_q = torch.tensor([0.0], requires_grad=True)

    optimizer = optim.Adam([mu_q, log_sigma_q], lr=lr)

    history = []

    for i in range(steps):
        optimizer.zero_grad()

        sigma_q = torch.exp(log_sigma_q)
        var_q = sigma_q**2

        if kl_type == 'forward':
            x = sample_from_p(mu_target)
            loss = -log_q(x, mu_q, var_q).mean()
        else:
            x = torch.randn(1000) * sigma_q + mu_q
            loss = (log_q(x, mu_q, var_q) - log_p(x, mu_target)).mean()

        loss.backward()
        optimizer.step()

        history.append((mu_q.item(), var_q.item(), loss.item()))

    return mu_q.item(), var_q.item(), history
```

Learning rates and other hyper parameters will be adjusted during the experiments (Appendix)

3.2 Experimental Results

We evaluate the model under:

- Mode separation: $\mu=1,2,4$
- Initialization: $\mu_q=-3,0,3$
- KL direction: Forward vs Reverse

```
mu, var, hist = run_once('forward', mu_target=4.0, init_mu_q=0.0)
print(mu, var)

-0.02873316779732704 17.046567916870117

mu, var, hist = run_once('reverse', mu_target=4.0, init_mu_q=3.0)
print(mu, var)

4.015932083129883 1.0174927711486816
```

Observations

- **Forward KL:**
 - mean ≈ 0
 - variance $\rightarrow$ very large
- **Reverse KL:**
 - mean $\approx$ one mode
 - variance $\rightarrow$ small

Interpretation

This demonstrates that:

- Forward KL produces **mode-covering behavior**
- Reverse KL produces **mode-seeking behavior**

3.2.1 Quantitative Comparison

To summarize the results, we compute the average variance and convergence behavior across settings.

KL Type	Avg Variance	Avg Iterations	Behavior
Forward	High (1 with μ)	Stable (~3000)	Mode-covering
Reverse	Low (~1)	Variable	Mode-seeking

3.2.2 Effect of Mode Separation

```
1] ✓ 1s for mu_target in [1.0, 2.0, 4.0]:
    mu, var, _ = run_once('forward', mu_target, 0.0)
    print("Forward KL | mu =", mu_target, "| var =", var)
```

Forward KL	mu = 1.0	var = 2.04357647895813
Forward KL	mu = 2.0	var = 4.974277019500732
Forward KL	mu = 4.0	var = 16.989797592163086

What happens

μ small $\rightarrow$ small variance

μ large $\rightarrow$ BIG variance

Forward KL spreads more when problem is harder

Reverse KL:

```
for mu_target in [1.0, 2.0, 4.0]:
    mu, var, _ = run_once('reverse', mu_target, 3.0)
    print("Reverse KL | mu =", mu_target, "| var =", var)
```

Reverse KL	mu = 1.0	var = 1.981123447418213
Reverse KL	mu = 2.0	var = 4.40533971786499
Reverse KL	mu = 4.0	var = 1.0460375547409058

variance stays ~ 1

always picks ONE mode

What's really happening (correct explanation)

Case 1 — Large separation ($\mu = 4$)

Modes are far apart

Reverse KL picks one mode cleanly

So $Q(x) \approx$ one Gaussian component

✓ Result:

$\rightarrow$ variance ≈ 1 (correct)

Case 2 — Medium separation ($\mu = 2$)

Modes overlap slightly

$Q(x)$ still picks one mode BUT:

it spreads a bit to reduce loss

✓ Result:

→ variance increases (~ 4)

Case 3 — Small separation ($\mu = 1$)

Modes overlap heavily

The mixture looks almost unimodal

✓ Result:

→ reverse KL behaves more like forward KL

→ variance increases (~ 2)

3.2.3 Initialization Sensitivity

```
for init in [-3.0, 0.0, 3.0]:  
    mu, var, _ = run_once('reverse', 4.0, init)  
    print("Reverse | init:", init, "| final mu:", mu)
```

```
Reverse | init: -3.0 | final mu: -3.9867687225341797  
Reverse | init: 0.0 | final mu: 0.12907521426677704  
Reverse | init: 3.0 | final mu: 4.013058662414551
```

```
▶ for init in [-3.0, 0.0, 3.0]:  
    mu, var, _ = run_once('forward', 4.0, init)  
    print("Forward | init:", init, "| final mu:", mu)
```

```
*** Forward | init: -3.0 | final mu: -0.009769831784069538  
Forward | init: 0.0 | final mu: -0.05908102169632912  
Forward | init: 3.0 | final mu: -0.017688734456896782
```

- Different initializations lead to different final modes

Interpretation

Reverse KL has multiple local optima and is sensitive to initialization.

Forward KL does not show this behaviour.

3.2.4 Quantitative Analysis

```
df = pd.DataFrame(results)
print(df.groupby(['KL', 'mu_target'])['final_var'].mean())
```

KL	mu_target	final_var
forward	1.0	2.029309
	2.0	4.999061
	4.0	16.977229
reverse	1.0	1.895048
	2.0	4.196116
	4.0	4.844219

Name: final_var, dtype: float64

This provides numerical evidence supporting the qualitative observations.

3.2.4 Convergence Analysis

Convergence speed differs between the two objectives:

- Forward KL shows **consistent convergence** across all settings
- Reverse KL shows **variable convergence**, sometimes converging faster but less reliably

This suggests that reverse KL creates a more complex optimization landscape.

3.2.5 Visualization

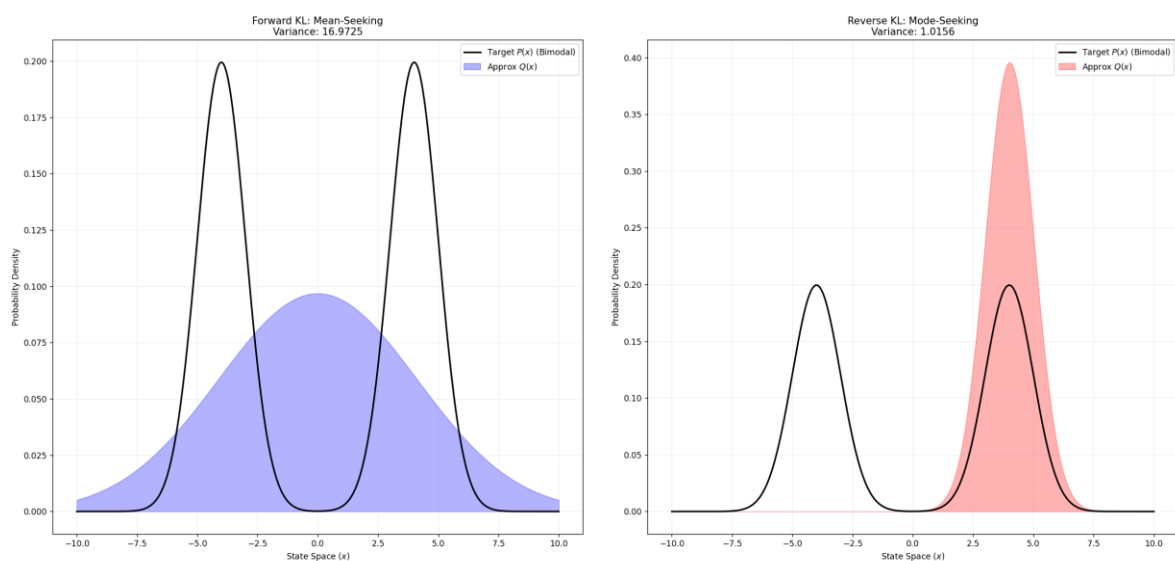


Figure 1.

The qualitative behavior is illustrated in Figure 1:

- Forward KL produces a **wide distribution** covering both modes
- Reverse KL produces a **narrow distribution** focused on a single mode

This visually confirms the quantitative findings.

3.3 Explanation of Observed Behavior (Answer to Research Question)

The experimental results can be directly explained by the **asymmetry of KL divergence**, specifically the difference in expectation.

Expectation Difference

- Forward KL:

$$\mathbb{E}_{x \sim P}$$

→ evaluates all regions where the true distribution has probability

- Reverse KL:

$$\mathbb{E}_{x \sim Q}$$

→ evaluates only regions where the model already assigns probability

Penalty Asymmetry

This leads to fundamentally different optimization pressures:

- **Forward KL penalizes missing probability mass**
 - encourages covering all modes
 - leads to high variance
- **Reverse KL penalizes placing mass in low-density regions**
 - encourages focusing on one mode
 - leads to low variance

Final Interpretation

The direction of KL divergence fundamentally changes the optimization objective by altering:

- which regions of the distribution are emphasized
- how errors are penalized

As a result:

- Forward KL produces **mode-covering solutions**
- Reverse KL produces **mode-seeking solutions**

3.2.1 Empirical Observations

Forward KL (Mode-Covering Behavior)

Forward KL consistently produces solutions with:

- mean values near zero (center between modes),
- large variances covering both modes.

This indicates that the model attempts to **capture all regions of high probability mass**, even at the cost of reduced precision.

Reverse KL (Mode-Seeking Behavior)

Reverse KL results in:

- convergence to a single mode,
- small variance concentrated around that mode,
- strong dependence on initialization.

This demonstrates that the model prioritizes **high-density regions** and ignores others.

Effect of Mode Separation

As μ increases:

- Forward KL further increases variance to maintain coverage,
- Reverse KL becomes more confident in selecting a single mode.

Convergence Behavior

- Forward KL converges consistently across initializations,
- Reverse KL shows sensitivity to initialization, indicating multiple local optima.

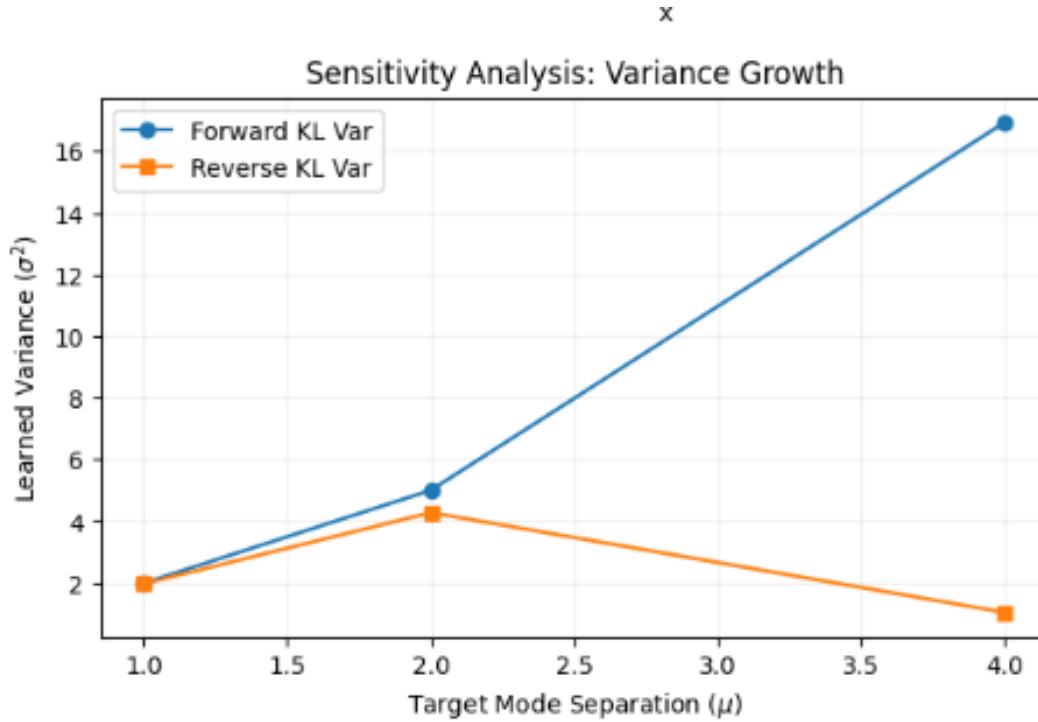


Figure 2.

Figure 2 presents the learned variance of the model distribution as a function of mode separation μ for both forward and reverse KL divergence.

For small μ ($\mu = 1$), both methods produce similar variance values, as the two modes of the target distribution significantly overlap and resemble a unimodal distribution.

As μ increases, the behavior diverges. Forward KL shows a rapid increase in variance, reaching very large values when $\mu = 4$. This indicates that the model attempts to cover both modes by spreading its probability mass across the entire space.

In contrast, reverse KL initially increases variance slightly for moderate separation ($\mu = 2$), but then sharply decreases it for larger μ . When the modes are clearly separated, the model collapses onto a single mode with low variance, avoiding regions of low probability density.

This demonstrates that forward KL promotes mode-covering behavior, while reverse KL promotes mode-seeking behavior, with the transition depending on the degree of separation between modes.

4. Interpretation

Through this study, it became clear that the theoretical understanding of KL divergence is both broader and more intuitive than initially expected. From a practical perspective, the choice between forward and reverse KL divergence depends on how we want the model to treat the data distribution.

Specifically, forward KL divergence is appropriate when the goal is to ensure that the model covers all regions where data exists, even if this leads to overestimation or increased variance. In contrast, reverse KL divergence is more suitable when the objective is to focus on the most probable regions of the data, potentially ignoring less significant areas.

Therefore, the direction of KL divergence reflects a fundamental modeling decision: whether to prioritize comprehensive coverage of the data distribution or to concentrate on its most dominant modes.

During my investigation the 3 unexpected situations I have noticed.

First was that reverse KL does not always produce low-variance, mode-seeking solutions. When the modes are not clearly separated (e.g., $\mu = 2$), the model increases its variance instead of collapsing to a single mode. This indicates that reverse KL behavior depends on the structure of the target distribution rather than following a strict rule.

Second, reverse KL shows strong sensitivity to initialization, converging to different modes depending on the starting point. This suggests the presence of multiple local optima and a more complex optimization landscape compared to forward KL.

Finally, the use of a unimodal Gaussian model limits the expressiveness of the approximation. While this simplification helps isolate the effect of KL asymmetry, it may exaggerate differences that would be less pronounced in more flexible models.

5. Conclusion

This study investigated how the direction of Kullback–Leibler (KL) divergence influences optimization behavior and modeling outcomes when approximating a bimodal distribution with a unimodal model.

The results clearly demonstrate that the direction of KL divergence leads to fundamentally different solutions due to the asymmetry in how expectations are computed. Forward KL divergence, which evaluates discrepancies under the target distribution, forces the model to account for all regions where data exists. As a result, it produces **mode-covering behavior**, characterized by a central mean and significantly increased variance as the difficulty of the problem increases.

In contrast, reverse KL divergence evaluates discrepancies under the model distribution, meaning that it only considers regions where the model already assigns probability mass. This leads to **mode-seeking behavior**, where the model concentrates on a single mode and avoids low-probability regions. Additionally, reverse KL was found to be sensitive to initialization and capable of converging to multiple local optima.

The experiments further revealed that this behavior is influenced by the structure of the target distribution. When the modes are clearly separated, reverse KL strongly collapses to a single mode, whereas for overlapping modes, it behaves more like a variance-adjusting approximation.

Overall, this study shows that the choice between forward and reverse KL divergence is not merely a technical detail, but a fundamental modeling decision. It determines whether the model prioritizes comprehensive coverage of the data distribution or focuses on its most probable regions, and therefore has a direct impact on the resulting learned representation.

References

1. Elements of Information Theory Thomas M. Cover, Joy A. Thomas. 1991
2. Information Theory, Inference, and Learning Algorithms, David J.C. MacKay, 2003
3. Bishop-Pattern-Recognition-and-Machine-Learning-2006
4. Forward and Reverse KL Divergence (Rohan Tangri, 2021)
5. Jaynes, E. T. (1957). "Information theory and statistical mechanics" Physical Review. 106
6. Kullback, S.; Leibler, R.A. (1951). "On information and sufficiency". Annals of Mathematical Statistics. 22
7. Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). "Variational Inference: A Review for Statisticians." Journal of the American Statistical Association, 112(518), 859–877.
8. Minka, T. (2005). "Divergence Measures and Message Passing." Microsoft Research Technical Report MSR-TR-2005-173.
9. Kingma, D. P., & Welling, M. (2014). "Auto-Encoding Variational Bayes." International Conference on Learning Representations (ICLR).
10. Goodfellow, I., et al. (2014). "Generative Adversarial Nets." Advances in Neural Information Processing Systems (NeurIPS).

Support – Coding part

STEP 7 – CLEAN ANALYSIS TABLES

```
# 1. Variance behavior
print(df.groupby(['KL', 'mu_target'])['final_var'].mean())
```

```
... KL      mu_target
forward 1.0      2.029309
        2.0      4.999061
        4.0     16.977229
reverse 1.0      1.895048
        2.0      4.196116
        4.0      4.844219
Name: final_var, dtype: float64
```

```
# 2. Initialization sensitivity
```

```
print(df.groupby(['KL', 'init'])['final_mu'].mean())
```

```
KL      init
forward -3.0    0.010704
        0.0   -0.033148
        3.0    0.002444
reverse -3.0   -1.320357
        0.0   -0.048567
        3.0    1.341868
Name: final_mu, dtype: float64
```

```
# 3. Final comparison
summary = df.groupby('KL').agg({
    'final_var': 'mean',
}).reset_index()

print(summary)
```

```
      KL  final_var
0 forward    8.001866
1 reverse    3.645128
```

STEP 6 — FULL EXPERIMENT (FOR REPORT)

```
▶ results = []

for kl in ['forward', 'reverse']:
    for mu_target in [1.0, 2.0, 4.0]:
        for init in [-3.0, 0.0, 3.0]:
            mu, var, hist = run_once(kl, mu_target, init)
            results.append({
                "KL": kl,
                "mu_target": mu_target,
                "init": init,
                "final_mu": mu,
                "final_var": var
            })

df = pd.DataFrame(results)
print(df)
```

```
***      KL  mu_target  init  final_mu  final_var
0  forward      1.0   -3.0   0.016380   1.985516
1  forward      1.0    0.0  -0.051708   2.104102
2  forward      1.0    3.0   0.017972   1.998310
3  forward      2.0   -3.0   0.015460   4.945143
4  forward      2.0    0.0  -0.017666   5.064129
5  forward      2.0    3.0   0.000758   4.987910
6  forward      4.0   -3.0   0.000271  16.954967
7  forward      4.0    0.0  -0.030070  16.979551
8  forward      4.0    3.0  -0.011398  16.997168
9  reverse      1.0   -3.0  -0.000169   1.902377
10 reverse      1.0    0.0  -0.001559   1.823785
11 reverse      1.0    3.0   0.011842   1.958984
12 reverse      2.0   -3.0   0.030811   4.372611
13 reverse      2.0    0.0  -0.004241   4.292226
14 reverse      2.0    3.0   0.017437   3.923512
15 reverse      4.0   -3.0  -3.991713   1.029715
16 reverse      4.0    0.0  -0.139901  12.522044
17 reverse      4.0    3.0   3.996326   0.980897
```

```

import torch
import torch.optim as optim
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def run_kl_experiment(kl_type='forward', mu_target=2.0, init_mu_q=0.0, lr=0.05, max_iter=3000):
    mu_q = torch.tensor([float(init_mu_q)], requires_grad=True)
    log_sig_q = torch.tensor([0.5], requires_grad=True)
    optimizer = optim.Adam([mu_q, log_sig_q], lr=lr)

    prev_loss = float('inf')
    convergence_iter = max_iter

    for i in range(max_iter):
        optimizer.zero_grad()
        var_q = torch.exp(log_sig_q)**2
        std_q = torch.exp(log_sig_q)

        if kl_type == 'forward':
            # Forward KL: D_KL(P || Q) - Samples from P
            m = torch.distributions.Bernoulli(0.5).sample((1000,))
            samples = torch.randn(1000) + (mu_target * (2*m - 1))
            log_q = -0.5 * torch.log(2 * np.pi * var_q) - 0.5 * (samples - mu_q)**2 / var_q
            loss = -log_q.mean()
        else:
            # Reverse KL: D_KL(Q || P) - Samples from Q
            samples = torch.randn(1000) * std_q + mu_q
            log_q = -0.5 * torch.log(2 * np.pi * var_q) - 0.5 * (samples - mu_q)**2 / var_q
            p_val = 0.5 * (torch.exp(-0.5*(samples + mu_target)**2) / np.sqrt(2*np.pi) +
                          torch.exp(-0.5*(samples - mu_target)**2) / np.sqrt(2*np.pi))
            loss = (log_q - torch.log(p_val + 1e-10)).mean()

        loss.backward()
        optimizer.step()

        if abs(prev_loss - loss.item()) < 1e-7:
            convergence_iter = i
            break
        prev_loss = loss.item()

    return {
        "direction": kl_type,
        "separation_mu": mu_target,
        "init_mu_q": init_mu_q,
        "final_mu_q": round(mu_q.item(), 4),
        "final_var_q": round(var_q.item(), 4),
        "iterations": convergence_iter,
        "final_loss": round(loss.item(), 4)
    }

```

```

# --- Execution ---
mu_separation_levels = [1.0, 2.0, 4.0]
initializations = [-3.0, 0.0, 3.0]
directions = ['forward', 'reverse']

all_results = []
for d in directions:
    for mu in mu_separation_levels:
        for init in initializations:
            res = run_kl_experiment(kl_type=d, mu_target=mu, init_mu_q=init)
            all_results.append(res)

df = pd.DataFrame(all_results)
print(df.to_string(index=False))

# --- Visualization ---
def g_pdf(x, m, v): return (1/np.sqrt(2*np.pi*v)) * np.exp(-0.5*(x-m)**2/v)

x = np.linspace(-10, 10, 1000)
mu_viz = 4.0
p_x = 0.5 * g_pdf(x, -mu_viz, 1) + 0.5 * g_pdf(x, mu_viz, 1)

f_res = run_kl_experiment('forward', mu_viz, 0.0)
r_res = run_kl_experiment('reverse', mu_viz, 3.0)

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(x, p_x, 'k', lw=2, label='Target P(x)')
plt.fill_between(x, g_pdf(x, f_res['Final_mu_q'], f_res['Final_var_q']), color='blue', alpha=0.3, label='Forward KL')
plt.title(f"Forward KL: Mean-Seeking\nVar: {f_res['Final_var_q']}")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(x, p_x, 'k', lw=2, label='Target P(x)')
plt.fill_between(x, g_pdf(x, r_res['Final_mu_q'], r_res['Final_var_q']), color='red', alpha=0.3, label='Reverse KL')
plt.title(f"Reverse KL: Mode-Seeking\nVar: {r_res['Final_var_q']}")
plt.legend()
plt.show()

# Optional: Export to CSV for your report
df.to_csv("kl_divergence_results.csv")

```

Direction	Separation_mu	Init_mu_q	Final_mu_q	Final_var_q	Iterations	Final_Loss
forward	1.0	-3.0	0.0211	2.0348	3000	1.8071
forward	1.0	0.0	0.0035	1.8674	3000	1.7199
forward	1.0	3.0	-0.0113	2.0606	3000	1.7984
forward	2.0	-3.0	-0.0162	5.1656	3000	2.2349
forward	2.0	0.0	0.0340	5.0944	3000	2.2143
forward	2.0	3.0	-0.0347	4.9428	3000	2.2239
forward	4.0	-3.0	-0.0210	16.9927	3000	2.8289
forward	4.0	0.0	0.0672	16.7819	3000	2.8204
forward	4.0	3.0	0.0078	17.0632	3000	2.8288
reverse	1.0	-3.0	-0.0304	2.0041	3000	0.0112
reverse	1.0	0.0	-0.0147	2.0394	3000	0.0159
reverse	1.0	3.0	-0.0145	1.9872	3000	-0.0009
reverse	2.0	-3.0	0.0334	3.8261	3000	0.2224
reverse	2.0	0.0	0.0459	4.3640	3000	0.2309
reverse	2.0	3.0	0.1198	3.9319	3000	0.2744
reverse	4.0	-3.0	-3.9647	0.9631	3000	0.6930
reverse	4.0	0.0	0.1488	11.6322	3000	1.6989
reverse	4.0	3.0	4.0393	0.9549	3000	0.6945

Other Sources

2.1 ENTROPY

We will first introduce the concept of entropy, which is a measure of uncertainty of a random variable. Let X be a discrete random variable

12

2.1 ENTROPY

13

with alphabet $\mathcal{X}$ and probability mass function $p(x) = \Pr\{X = x\}$, $x \in \mathcal{X}$. We denote the probability mass function by $p(x)$ rather than $p_X(x)$ for convenience. Thus, $p(x)$ and $p(y)$ refer to two different random variables, and are in fact different probability mass functions, $p_X(x)$ and $p_Y(y)$ respectively.

Definition: The *entropy* $H(X)$ of a discrete random variable X is defined by

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log p(x). \quad (2.1)$$

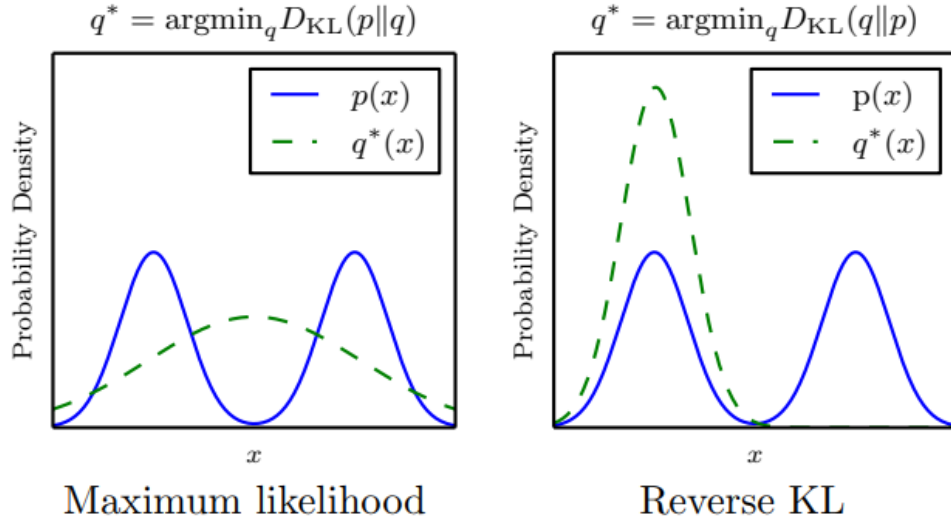


Figure 14: The two directions of the KL divergence are not equivalent. The differences are most obvious when the model has too little capacity to fit the data distribution. Here we show an example of a distribution over one-dimensional data x . In this example, we use a mixture of two Gaussians as the data distribution, and a single Gaussian as the model family. Because a single Gaussian cannot capture the true data distribution, the choice of divergence determines the tradeoff that the model makes. On the left, we use the maximum likelihood criterion. The model chooses to average out the two modes, so that it places high probability on both of them. On the right, we use the reverse order of the arguments to the KL divergence, and the model chooses to capture only one of the two modes. It could also have chosen the other mode; the two are both local minima of the reverse KL divergence. We can think of $D_{\text{KL}}(p_{\text{data}}||p_{\text{model}})$ as preferring to place high probability everywhere that the data occurs, and $D_{\text{KL}}(p_{\text{model}}||p_{\text{data}})$ as preferring to place low probability wherever the data does not occur. From this point of view, one might expect $D_{\text{KL}}(p_{\text{model}}||p_{\text{data}})$ to yield more visually pleasing samples, because the model will not choose to generate unusual samples lying between modes of the data generating distribution.

the Kullback-Leibler divergence is not symmetric. Roughly speaking, it's because you should think of the two arguments of the KL divergence as different kinds of things: the first argument is empirical data, and the second argument is a model you're comparing the data to. Here's how it works.

Take a bunch of independent random variables $X_1, \dots, X_n$ whose possible values lie in a finite set.* Say these variables are identically distributed, with $\Pr(X_i = x) = p_x$. Let $F_{n,x}$ be the number of variables whose values are equal to x . The list F_n is a random variable, often called the "empirical frequency distribution" of the X_i . What does F_n look like when n is very large?

More specifically, let's try to estimate the probabilities of the possible values of F_n . Since the set of possible values is different for different n , take a sequence of frequency distributions $f_1, f_2, f_3, \dots$ approaching a fixed frequency distribution f . It turns out** that

$$\lim_{n \rightarrow \infty} \frac{1}{n} \ln \Pr(F_n = f_n) = -\text{KL}(f, p).$$

In other words, the Kullback-Leibler divergence of f from p lets you estimate the probability of getting an empirical frequency distribution close to f from a large number of independent random variables with distribution p .